

@code.withnarayan

Learn
Practice
Build
Grow

Better
Code
Brighter
Future

JavaScript

COMPLETE HANDWRITTEN NOTES

— Zero to Hero —

- ✓ Easy to Understand
- ✓ Concepts with Examples
- ✓ Diagrams & Visuals
- ✓ Interview Questions
- ✓ Quick Revision
- ✓ Beginner to Advanced



"Small
Steps
Big
Progress"



Code
Logic
Creativity
Problem Solving
= Better You



Write
Test
Debug
Improve

One Language
Endless Possibilities

1. JavaScript Introduction

What is JavaScript ?

- JavaScript is a high-level, interpreted programming language.
- It is mainly used to make web pages interactive and dynamic.
- It can run in a web browser as well as on servers (using Node.js).

Why Learn JavaScript ?

- ✓ Most popular programming language
- ✓ Essential for web development
- ✓ Used in frontend, backend, mobile apps, desktop apps
- ✓ Large community and job opportunities
- ✓ Helps in learning other frameworks (React, Node.js, etc.)

Where is JavaScript Used ?

- Web Development (Frontend) → HTML + CSS + JS
- Web Development (Backend) → Node.js
- Mobile App Development → React Native
- Desktop App Development → Electron
- Game Development → Phaser, Three.js
- Web Servers → Node.js
- Browser Extensions → JavaScript

Simple Example

```
// This is a simple JavaScript program
console.log("Hello, JavaScript!");
console.log("Welcome to @code.withnarayan");
```

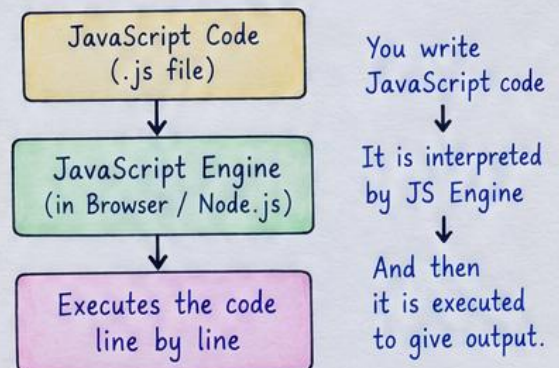
Key Facts

- Developed by : Brendan Eich
- First released : 1995
- Official name : ECMAScript
- Language type : Scripting language
- Runs on : Browser (Client-side) & Node.js (Server-side)
- File extension : .js

Features of JavaScript

- Lightweight and easy to learn
- Interpreted (no need to compile)
- Dynamic typing
- Event-driven
- Supports object-oriented, functional and procedural styles
- Cross-platform (runs on any OS)

How JavaScript Runs ?



Output (in Console)

```
Hello, JavaScript!
Welcome to @code.withnarayan
```

Conclusion

- JavaScript is the backbone of modern web development.
- It is simple, powerful and full of opportunities.
- Master JavaScript and you can build amazing things!

"JavaScript
turns ideas into
interactive reality."

2. Variables & Data Types

What is a Variable?

- A variable is a named storage location used to store data in JavaScript.
- We can store different types of data in a variable and use it later.
- In JavaScript, we do not need to specify the data type explicitly (it is dynamically typed).

Declaring Variables

```
var x = 10; // old way (function scoped)
let y = 20; // block scoped
const z = 30; // constant (cannot be reassigned)
```

Example:

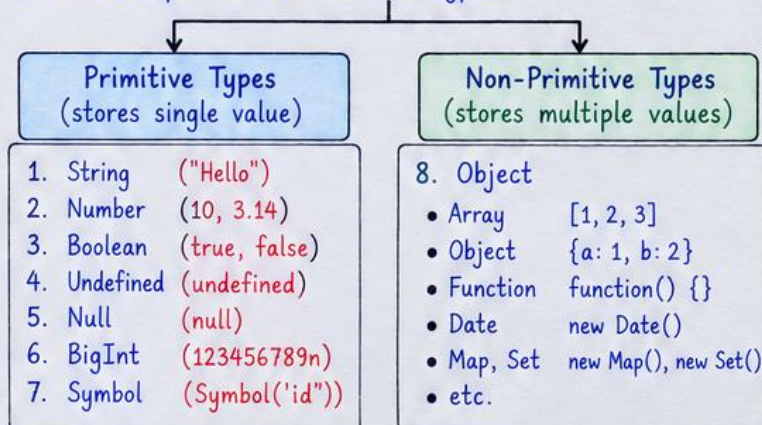
```
let name = "Narayan";
const pi = 3.14;
var isStudent = true;
```

Difference between var, let and const

Feature	var	let	const
Scope	Function	Block	Block
Can be redeclared?	Yes	No	No
Can be reassigned?	Yes	Yes	No
Hoisting	Yes	Yes (temporal dead zone)	Yes (temporal dead zone)

Data Types in JavaScript

- JavaScript has 8 main data types.



Check Data Type

```
console.log(typeof "Hello"); // string
console.log(typeof 10); // number
console.log(typeof true); // boolean
console.log(typeof undefined); // undefined
console.log(typeof null); // object (special case)
console.log(typeof {}); // object
console.log(typeof []); // object
```

Rules for Naming Variables

- Can contain letters, numbers, _ and \$
- Cannot start with a number
- Cannot use reserved keywords (e.g. let, const, var, if, else, etc.)
- Case sensitive (age ≠ Age)
- Use meaningful names

Valid Examples:

```
let userName = "Narayan";
let _count = 0;
let $price = 100;
```

Invalid Examples:

```
let 1name = "JS"; // cannot start with number
let user-name = "JS"; // hyphen not allowed
let let = 10; // reserved keyword
```

Example with Different Data Types

```
let str = "JavaScript";
let num = 100;
let bool = false;
let und; // undefined
let n = null;
let big = 123456789n;
let sym = Symbol('id');
let arr = [1, 2, 3];
let obj = { name: "Narayan", age: 21 };
console.log(typeof obj); // object
```

Key Takeaways

- ✓ Use let and const (avoid var).
- ✓ JavaScript is dynamically typed.
- ✓ There are 8 main data types.
- ✓ Use meaningful variable names.
- ✓ Always check data type using typeof.

3. Operators in JavaScript

Operators are used to perform operations on variables and values.

1 Arithmetic Operators

Used to perform mathematical operations.

Operator	Meaning	Example	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	6 / 3	2
%	Modulus (Remainder)	5 % 2	1
**	Exponentiation	2 ** 3	8

2 Assignment Operators

Used to assign values to variables.

Operator	Example	Same as
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
**=	x **= 3	x = x ** 3

3 Comparison Operators

Used to compare two values. It returns a boolean value (true/false).

Operator	Meaning	Example	Result
==	Equal (value)	5 == '5'	true
===	Equal (value & type)	5 === '5'	false
!=	Not equal	5 != 3	true
!==	Not equal (value & type)	5 !== '5'	true
>	Greater than	5 > 3	true
<	Less than	5 < 3	false
>=	Greater than or equal	5 >= 5	true
<=	Less than or equal	3 <= 2	false

4 Logical Operators

Used to combine multiple conditions.

Operator	Meaning	Example	Result
&&	AND (both true)	true && false	false
	OR (any true)	true false	true
!	NOT (inverse)	!true	false

5 Ternary Operator

A short form of if-else statement.

```
let age = 18;  
let status = (age >= 18) ? "Adult" : "Minor";  
console.log(status); // Output: Adult
```

6 Increment & Decrement Operators

Used to increase or decrease the value by 1.

Operator	Meaning	Example	Result
++	Increment (pre)	let x = ++a	(a = a + 1)
++	Increment (post)	let x = a++	(a = a + 1)
--	Decrement (pre)	let x = --a	(a = a - 1)
--	Decrement (post)	let x = a--	(a = a - 1)

7 Operator Precedence

- Operators are evaluated based on precedence (priority).
- Parentheses () have the highest priority.
- If two operators have the same precedence, evaluation happens from left to right (except for some like **).

Example:

```
let result = 2 + 3 * 4;  
console.log(result); // Output: 14  
// (Multiplication happens first)
```

Key Takeaways

- ✓ Operators help in performing operations on data.
- ✓ Use === instead of == for accurate comparison.
- ✓ Use parentheses () to avoid confusion.
- ✓ Ternary operator is useful for short conditions.

5. Type Conversion & Type Coercion

Type conversion means changing a value from one data type to another.

JavaScript provides both **explicit** conversion (manual) and **implicit** conversion (automatic).

1. Explicit Type Conversion

- We manually convert the data type using built-in functions.
- More predictable and recommended.

Function	Converts to
String(value)	String
Number(value)	Number
Boolean(value)	Boolean
parseInt(value)	Integer (Number)
parseFloat(value)	Float (Number)

Examples (Explicit)

```
let num = 10;
let str = String(num);    // "10"
console.log(typeof str);  // string

let n = "123";
let x = Number(n);        // 123
console.log(typeof x);    // number

let a = "10.5";
console.log(parseInt(a)); // 10
console.log(parseFloat(a)); // 10.5

let b = 1;
let c = Boolean(b);
console.log(c);           // true
```

2. Implicit Type Conversion (Type Coercion)

- JavaScript automatically converts one type to another when needed.
- Happens during operations like +, -, ==, etc.
- Can be useful but sometimes confusing.

Examples (Implicit)

```
console.log("5" + 2);    // "52" (string)
console.log("5" - 2);    // 3 (number)
console.log(true + 1);   // 2
console.log(false + 1);  // 1
console.log("10" == 10); // true (type coerced)
console.log("" == 0);    // true
console.log(null == undefined); // true
```

Common Coercion Examples

```
console.log(1 + "2");    // "12"
console.log(2 + "2" + 2); // "222"
console.log(2 + 2 + "2"); // "42"

console.log([] + []);    // "" (empty string)
console.log([] + {});    // "[object Object]"
console.log({} + []);    // 0
console.log({} + {});    // NaN

console.log(true == 1);  // true
console.log(false == 0); // true
console.log("" == false); // true
```

Points to Remember

- ✓ Use explicit conversion when you need correct and predictable results.
- ✓ Implicit conversion can lead to unexpected results.
- ✓ The + operator behaves differently with strings.
- ✓ Use === instead of == to avoid type coercion.
- ✓ Be careful with [], {}, null, undefined in comparisons.

Quick Examples

```
console.log(Number(""));    // 0
console.log(Boolean(""));   // false
console.log(Boolean("Narayan")); // true
console.log(String(true));  // "true"
console.log(!0);           // false
console.log(!1);           // true
```

6. Conditional Statements

Conditional statements are used to execute a block of code based on a condition. They help in decision making in JavaScript.

1. if Statement

- Executes a block of code if the condition is true.

```
let age = 18;
if (age >= 18) {
  console.log("You are an adult");
}
```

2. if - else Statement

- Executes one block if condition is true and another block if it is false.

```
let age = 16;
if (age >= 18) {
  console.log("Adult");
} else {
  console.log("Not an adult");
}
```

3. else if Ladder

- Used to check multiple conditions.

```
let marks = 75;
if (marks >= 90) {
  console.log("Grade A");
} else if (marks >= 75) {
  console.log("Grade B");
} else if (marks >= 60) {
  console.log("Grade C");
} else {
  console.log("Grade D");
}
```

4. Nested if Statement

- An if statement inside another if statement.

```
let age = 20;
let hasID = true;

if (age >= 18) {
  if (hasID) {
    console.log("You can vote");
  } else {
    console.log("ID is required");
  }
} else {
  console.log("You are not eligible");
}
```

5. switch Statement

- Used to execute one block of code from multiple options.
- Useful when we have many conditions for the same variable.

```
let day = 3;
switch (day) {
  case 1:
    console.log("Monday");
    break;
  case 2:
    console.log("Tuesday");
    break;
  case 3:
    console.log("Wednesday");
    break;
  default:
    console.log("Invalid day");
}
```

6. Ternary Operator

- A shorthand way of writing if-else statement.

```
let age = 18;
let result = (age >= 18) ? "Adult" : "Minor";
console.log(result);
```

Key Takeaways

- ✓ Use if when only one condition is needed.
- ✓ Use else if for multiple conditions.
- ✓ Use switch for multiple fixed values.
- ✓ Ternary operator is useful for simple conditions.
- ✓ Write clear and readable conditions.

7. Loops in JavaScript

Loops are used to execute a block of code repeatedly until a certain condition is met.

1. for Loop

- Used when we know the number of iterations in advance.
- It has initialization, condition and update in a single line.

```
for (let i = 1; i <= 5; i++) {  
  console.log(i); // 1 2 3 4 5  
}
```

2. while Loop

- Checks the condition before executing the block.
- Useful when the number of iterations is not known.

```
let i = 1;  
while (i <= 5) {  
  console.log(i); // 1 2 3 4 5  
  i++;  
}
```

3. do...while Loop

- Executes the block at least once, then checks the condition.
- Useful when we want to run the code minimum one time.

```
let i = 1;  
do {  
  console.log(i); // 1 2 3 4 5  
  i++;  
} while (i <= 5);
```

4. for...in Loop

- Used to iterate over the properties (keys) of an object.

```
let person = {  
  name: "Narayan",  
  age: 21,  
  city: "Bhopal"  
};  
for (let key in person) {  
  console.log(key + " : " + person[key]);  
}  
// name : Narayan age : 21 city : Bhopal
```

5. for...of Loop

- Used to iterate over the values of an iterable object like array, string, etc.

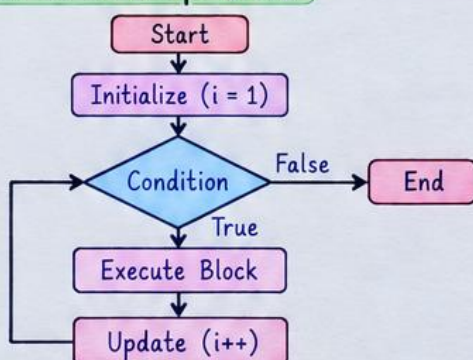
```
let fruits = ["Apple", "Banana", "Mango"];  
for (let fruit of fruits) {  
  console.log(fruit);  
}  
// Apple  
// Banana  
// Mango
```

6. Loop Control Statements

- **break** → used to exit the loop early.
- **continue** → used to skip the current iteration and move to the next.

```
for (let i = 1; i <= 5; i++) {  
  if (i == 3) {  
    continue; // skip 3  
  }  
  console.log(i); // 1 2 4 5  
}
```

Flow of a Loop (for)



Key Takeaways

- ✓ Use **for** loop when number of iterations is known.
- ✓ Use **while** loop when number of iterations is not known.
- ✓ Use **do...while** when you want to execute at least once.
- ✓ Use **for...in** for object properties (keys).
- ✓ Use **for...of** for iterable values (arrays, strings, etc.).
- ✓ Use **break** and **continue** to control the loop flow.

8. Functions in JavaScript

Functions are reusable blocks of code that perform a specific task. They help in writing modular and maintainable code.

1. Function Declaration

- A named function that can be called before its definition (due to hoisting).

```
function greet(name) {  
  console.log("Hello, " + name + "!");  
}  
greet("Narayan"); // Hello, Narayan!
```

2. Function Expression

- A function assigned to a variable.
- Cannot be called before its definition.

```
const greet = function(name) {  
  console.log("Hello, " + name + "!");  
};  
greet("Narayan"); // Hello, Narayan!
```

3. Arrow Function (ES6)

- Shorter syntax for writing functions.
- Does not have its own [this](#).

```
const add = (a, b) => {  
  return a + b;  
};  
console.log(add(5, 3)); // 8
```

4. Function Parameters

- Parameters are values passed to a function.
- A function can have multiple parameters.

```
function sum(a, b) {  
  return a + b;  
}  
console.log(sum(10, 20)); // 30
```

5. Default Parameters (ES6)

- Provide default values if no argument is passed.

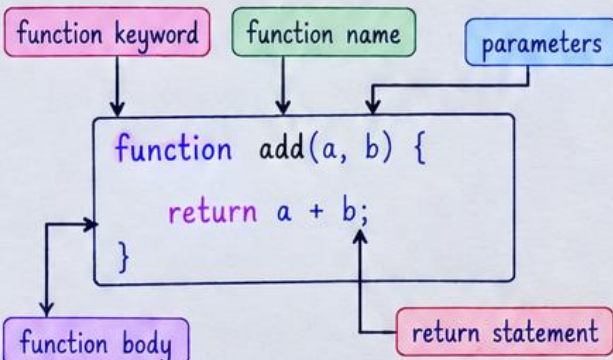
```
function greet(name = "Guest") {  
  console.log("Hello, " + name + "!");  
}  
greet(); // Hello, Guest  
greet("Narayan"); // Hello, Narayan
```

6. Return Statement

- Used to return a value from a function.
- If no return is specified, the function returns undefined.

```
function multiply(a, b) {  
  return a * b;  
}  
let result = multiply(4, 5);  
console.log(result); // 20
```

7. Function Anatomy



8. Key Takeaways

- ✓ Functions make code reusable.
- ✓ Function declaration is hoisted.
- ✓ Function expression is not hoisted.
- ✓ Arrow functions provide shorter syntax.
- ✓ Use meaningful function names.
- ✓ Parameters allow dynamic input.
- ✓ Use default parameters to handle missing values.
- ✓ Use return to get a value from a function.

9. Arrays in JavaScript

An array is a special variable that can store multiple values in a single variable.
Arrays can store elements of different data types.

1. Creating an Array

- Using array literal (recommended)

```
let fruits = ["Apple", "Banana", "Mango"];  
console.log(fruits); // ["Apple", "Banana", "Mango"]
```
- Using new keyword

```
let nums = new Array(1, 2, 3, 4);  
console.log(nums); // [1, 2, 3, 4]
```

2. Array Indexing

- Array elements are accessed using index (index starts from 0).
- We can also update elements using index.

```
let fruits = ["Apple", "Banana", "Mango"];  
console.log(fruits[0]); // Apple  
console.log(fruits[1]); // Banana  
fruits[1] = "Orange";  
console.log(fruits); // ["Apple", "Orange", "Mango"]
```

fruits →

"Apple"	"Banana"	"Mango"
0	1	2

3. Array Properties

- `length` → returns the number of elements.
- Arrays are mutable (can be changed).

```
let nums = [10, 20, 30, 40];  
console.log(nums.length); // 4  
nums[2] = 100;  
console.log(nums); // [10, 20, 100, 40]
```

4. Iterating Over an Array

- Using for loop

```
let nums = [1, 2, 3, 4];  
for (let i = 0; i < nums.length; i++) {  
  console.log(nums[i]);  
}
```
- Using for...of loop (ES6)

```
for (let num of nums) {  
  console.log(num);  
}
```

5. Common Array Methods

Method	Description	Example	Result
<code>push()</code>	Add element at end	<code>arr.push(4)</code>	<code>[1,2,3,4]</code>
<code>pop()</code>	Remove last element	<code>arr.pop()</code>	removes 3
<code>unshift()</code>	Add element at start	<code>arr.unshift(0)</code>	<code>[0,1,2,3]</code>
<code>shift()</code>	Remove first element	<code>arr.shift()</code>	removes 1
<code>slice()</code>	Return a portion	<code>arr.slice(1,3)</code>	<code>[2,3]</code>
<code>splice()</code>	Add/Remove elements	<code>arr.splice(1,1,5)</code>	<code>[1,5,3]</code>
<code>indexOf()</code>	Return index	<code>arr.indexOf(3)</code>	2
<code>includes()</code>	Check if element exists	<code>arr.includes(2)</code>	true
<code>map()</code>	Create new array	<code>arr.map(x => x*2)</code>	<code>[2,4,6]</code>
<code>filter()</code>	Filter elements	<code>arr.filter(x => x > 2)</code>	<code>[3]</code>
<code>reduce()</code>	Reduce to single value	<code>arr.reduce((a,b) => a+b)</code>	6

6. Multi-type Array

- An array can store different data types.

```
let mixed = [1, "Hello", true, 3.5];  
console.log(mixed);  
// [1, "Hello", true, 3.5]
```

7. Key Takeaways

- ✓ Arrays store multiple values.
- ✓ Index starts from 0.
- ✓ Use `length` to get the size.
- ✓ Arrays are mutable.
- ✓ Many built-in methods are available.
- ✓ Arrays can store different data types.

10. Strings in JavaScript

A string is a sequence of characters used to represent text. In JavaScript, strings can be written using single quotes, double quotes or backticks (template literals).

1. Creating a String

- Strings can be created using single quotes ('), double quotes ("), or backticks (`).

```
let a = 'Hello';
let b = "JavaScript";
let c = `Welcome to JS`;

console.log(a, b, c);
```

3. String Indexing

- Characters in a string are accessed using index (starting from 0).
- We can also use bracket notation.

```
let str = "Narayan";
console.log(str[0]);           // N
console.log(str[3]);           // a
console.log(str[str.length - 1]); // n
```

5. Template Literals (ES6)

- Template literals use backticks (`).
- They allow string interpolation and multi-line strings.

```
let name = "Narayan";
let age = 21;
let msg = `My name is ${name}
and I am ${age} years old.`;

console.log(msg);
```

6. String Concatenation

- Strings can be concatenated using + operator or template literals.

```
let first = "Code";
let second = "With Narayan";
let result1 = first + " " + second;
console.log(result1); // Code With Narayan

let result2 = `${first} ${second}`;
console.log(result2); // Code With Narayan
```

2. String Length

- The length property returns the number of characters in a string.

```
let str = "Narayan";
console.log(str.length); // 7
```

4. Common String Methods

Method	Description	Example
toUpperCase()	Converts to uppercase	str.toUpperCase()
toLowerCase()	Converts to lowercase	str.toLowerCase()
trim()	Removes spaces (start/end)	str.trim()
slice()	Extracts a portion	str.slice(1, 4)
substring()	Extracts a portion	str.substring(1, 4)
replace()	Replaces a value	str.replace("a", "b")
includes()	Checks if substring exists	str.includes("JS")
startsWith()	Checks starting characters	str.startsWith("Na")
endsWith()	Checks ending characters	str.endsWith("an")
charAt()	Returns character at index	str.charAt(2)
split()	Converts to array	str.split(",")

7. split() and join()

- split() converts a string into an array.
- join() converts an array into a string.

```
let str = "a,b,c,d";
let arr = str.split(",");
console.log(arr); // ["a", "b", "c", "d"]

let newStr = arr.join("-");
console.log(newStr); // a-b-c-d
```

8. Key Takeaways

- ✓ Strings can be written using ' ', " " or ` `.
- ✓ Use length to get the size.
- ✓ Many useful built-in methods are available.
- ✓ Template literals make string handling easier.
- ✓ Strings are immutable (cannot be changed).
- ✓ Use split() and join() to convert between strings and arrays.

11. Objects in JavaScript

An object is a collection of key-value pairs used to store related data and functions (methods) in JavaScript.

1. Creating an Object

- Objects can be created using object literal (recommended) or using `new Object()`.

```
let student = {
  name: "Narayan",
  age: 21,
  branch: "CS",
  isStudent: true
};
```

2. Accessing and Modifying Properties

- We can access properties using dot notation or bracket notation.
- We can also add or update properties.

```
let student = { name: "Narayan", age: 21 };
console.log(student.name);    // Narayan
console.log(student["age"]);  // 21
student.city = "Raipur";
student.age = 22;
console.log(student);
```

3. Adding and Deleting Properties

- We can add new properties at any time.
- We can delete properties using `delete` keyword.

```
let user = { name: "Narayan" };
user.email = "narayan@gmail.com"; // add
delete user.name;                  // delete
console.log(user);
```

4. Object Methods

- Objects can also have functions as properties (called methods).
- Methods can use `this` keyword to access object properties.

```
let person = {
  name: "Narayan",
  age: 21,
  greet: function() {
    console.log("Hello, I am " + this.name);
  }
};
person.greet(); // Hello, I am Narayan
```

5. Iterating Over an Object

- We can use `for...in` loop to iterate over object properties.
- `Object.keys()`, `Object.values()` and `Object.entries()` are also useful.

```
let student = { name: "Narayan", age: 21, city: "Raipur" };
// using for...in
for (let key in student) {
  console.log(key + " : " + student[key]);
}

// using Object.keys()
console.log(Object.keys(student)); // [name, age, city]
```

6. Nested Objects

- An object can contain another object as a property.

```
let student = {
  name: "Narayan",
  age: 21,
  address: {
    city: "Raipur",
    state: "Chhattisgarh"
  }
};
console.log(student.address.city); // Raipur
```

7. Key Takeaways

- ✓ Objects store data in key-value pairs.
- ✓ Properties can be accessed using dot or bracket notation.
- ✓ We can add, update and delete properties.
- ✓ Objects can have methods.
- ✓ Use `this` keyword inside methods.
- ✓ Use `for...in`, `Object.keys()`, `Object.values()` to iterate over objects.
- ✓ Objects can be nested to store complex data.

12. Error Handling in JavaScript

Error handling is used to handle runtime errors so that the program does not crash and we can show a meaningful message.

1. try...catch

- Used to catch and handle errors.
- Code that may throw an error is written in try block and the error is handled in catch block.

```
try {
  let a = 10 / 0;
  console.log(a);
} catch (error) {
  console.log("Error:", error.message);
}
```

2. finally Block

- The finally block always executes (whether an error occurs or not).
- Used for cleanup code (closing files, releasing resources, etc.).

```
try {
  let a = 10 / 0;
  console.log(a);
} catch (error) {
  console.log("Error:", error.message);
} finally {
  console.log("This will always run");
}
```

3. throw Keyword

- Used to manually throw an error.
- We can throw a string, number or an Error object.

```
let age = -5;
if (age < 0) {
  throw new Error("Age cannot be negative");
}
console.log("Valid age:", age);
```

4. Error Object

- JavaScript provides a built-in Error object with useful properties.

```
try {
  throw new Error("Something went wrong");
} catch (error) {
  console.log("Message:", error.message);
  console.log("Name:", error.name);
  console.log("Stack:", error.stack);
}
// Message: Something went wrong
// Name: Error
// Stack: (stack trace info)
```

5. Custom Error

- We can create our own error by extending the Error class.

```
class MyError extends Error {
  constructor(message) {
    super(message);
    this.name = "MyError";
  }
}

throw new MyError("This is a custom error");
```

6. Common Error Types

Error Type	Description
SyntaxError	Invalid syntax in code
ReferenceError	Using a variable that is not defined
TypeError	Operation on wrong data type
RangeError	Value is not in allowed range
URIError	Error in encodeURI() or decodeURI()
EvalError	Error in eval() function (rare)

7. Key Takeaways

- ✓ Use try...catch to handle errors.
- ✓ Use finally for cleanup code.
- ✓ Use throw to raise custom errors.
- ✓ Error object provides useful information (message, name, stack).
- ✓ Different types of errors exist (SyntaxError, TypeError, etc.).
- ✓ Proper error handling makes the code more reliable and user-friendly.

13. ES6+ Features in JavaScript

ES6 (ECMAScript 2015) introduced many powerful features to make JavaScript more modern, clean and easier to write.

1. let, const and var

- var is function scoped.
- let and const are block scoped.
- Use const for values that do not change.

```
var x = 10;      // function scoped
let y = 20;      // block scoped
const z = 30;    // cannot be reassigned

console.log(x, y, z); // 10 20 30
```

2. Arrow Functions

- Arrow functions provide a shorter syntax.
- They do not have their own this.
- Useful for writing concise functions.

```
const add = (a, b) => {
  return a + b;
};
console.log(add(5, 3)); // 8
// single line (implicit return)
const square = n => n * n;
console.log(square(4)); // 16
```

3. Destructuring

- Allows us to extract values from arrays and objects easily.
- Makes the code more readable.

```
// Array destructuring
const [a, b] = [10, 20];
console.log(a, b); // 10 20

// Object destructuring
const person = { name: "Narayan", age: 21 };
const { name, age } = person;
console.log(name, age); // Narayan 21
```

4. Spread and Rest Operator

- Spread (...) is used to expand elements.
- Rest (...) is used to collect elements into an array.

```
// Spread operator
const arr1 = [1, 2, 3];
const arr2 = [...arr1, 4, 5];
console.log(arr2); // [1, 2, 3, 4, 5]

// Rest operator
function sum(...nums) {
  return nums.reduce((a, b) => a + b, 0);
}
console.log(sum(1, 2, 3, 4)); // 10
```

5. Template Literals

- Use backticks (`) instead of quotes.
- Supports multi-line strings and expression interpolation.

```
let name = "Narayan";
let age = 21;
let msg = `Hello, I am ${name} and
           I am ${age} years old.`;
console.log(msg);
```

6. Default Parameters

- Provide default values for function parameters.
- If no value is passed, the default value is used.

```
function greet(name = "Guest") {
  return `Hello, ${name}!`;
}
console.log(greet()); // Hello, Guest
console.log(greet("Narayan")); // Hello, Narayan
```

7. Key Takeaways

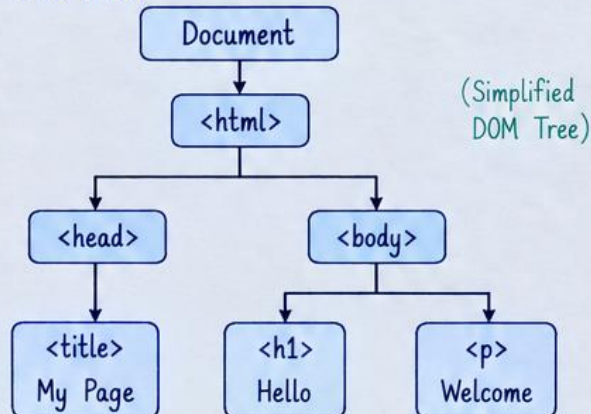
- ✓ ES6+ features make JavaScript more powerful and easier to write.
- ✓ Use let and const instead of var.
- ✓ Arrow functions provide shorter syntax.
- ✓ Destructuring helps to extract data easily.
- ✓ Spread and rest operators are very useful.
- ✓ Template literals make string handling better.
- ✓ Default parameters help to handle missing arguments.
- ✓ These features are widely used in modern JavaScript development.

14. DOM Manipulation in JavaScript

The Document Object Model (DOM) represents an HTML document as a tree of objects. JavaScript can be used to access, modify, add or delete HTML elements.

1. What is DOM?

- DOM is a programming interface for HTML documents.
- It represents the document as a tree structure.
- Each element, attribute and text is a node (object).
- We can use JavaScript to access and manipulate these nodes.



2. Selecting Elements

- We can select elements using different methods.
- Common methods:

Method	Description	Example
<code>getElementById()</code>	Selects element by id	<code>document.getElementById("id")</code>
<code>getElementsByName()</code>	Selects by class name	<code>document.getElementsByName("class")</code>
<code>getElementsByTagName()</code>	Selects by tag name	<code>document.getElementsByTagName("p")</code>
<code>querySelector()</code>	Selects first matching element (CSS selector)	<code>document.querySelector(".box")</code>
<code>querySelectorAll()</code>	Selects all matching elements (CSS selector)	<code>document.querySelectorAll(".box")</code>

```

let title = document.getElementById("main");
console.log(title); // element with id 'main'
let items = document.getElementsByClassName("item");
console.log(items); // HTMLCollection
let first = document.querySelector("h1");
console.log(first); // first <h1> element
  
```

3. Changing Content

- We can change the content of an element using `innerHTML`, `innerText` or `textContent`.

```

let heading = document.getElementById("title");
heading.innerHTML = "<b>Welcome</b>"; // HTML
heading.innerText = "Welcome JS"; // only text
heading.textContent = "Hello World"; // only text
  
```

5. Creating and Adding Elements

- We can create new elements using `createElement()` and add them using `appendChild()` or `append()`.

```

let newPara = document.createElement("p");
newPara.innerText = "This is a new paragraph";
document.getElementById("content").appendChild(newPara);

// or
document.getElementById("content").append(newPara);
  
```

4. Changing Styles

- We can modify CSS styles using the `style` property.

```

let box = document.getElementById("box");
box.style.color = "red";
box.style.backgroundColor = "yellow";
box.style.fontSize = "20px";
  
```

6. Removing Elements

- We can remove an element using `remove()`.

```

let para = document.getElementById("removeMe");
para.remove(); // removes the element
  
```

7. Useful DOM Properties

- `innerHTML` - gets/sets HTML content
- `innerText` - gets/sets text content
- `textContent` - gets/sets text content (faster)
- `style` - gets/sets inline styles
- `classList` - add/remove/toggle classes
- `children` - returns child elements
- `parentElement` - returns parent element

8. Key Takeaways

- ✓ DOM allows JavaScript to interact with HTML elements.
- ✓ We can select, modify, add and delete elements.
- ✓ Common selection methods: `getElementById()`, `querySelector()`, etc.
- ✓ Use `innerHTML` / `innerText` to change content.
- ✓ Use `style` to modify CSS styles.
- ✓ Use `createElement()` and `append()` to add elements.
- ✓ Use `remove()` to delete elements.
- ✓ DOM manipulation is widely used in web development.

15. Events & Event Handling in JavaScript

Events are actions that occur in the browser, usually as a result of user interaction with a web page. We can handle these events using event listeners.

1. What are Events ?

- Events are actions or occurrences that happen in the browser (e.g., click, mouse move, key press).
- They allow us to make web pages interactive.
- We can handle events using event listeners.
- An event can be triggered by user actions or by the browser itself.

3. Adding Event Listeners

- We can use `addEventListener()` to handle events.
- It allows multiple event handlers on the same element.

```
<button id="btn">Click Me</button>
<script>
  const btn = document.getElementById("btn");
  btn.addEventListener("click", function() {
    console.log("Button clicked!");
  });
</script>
```

5. Inline Event Handling

- We can also handle events directly in HTML using inline event attributes.
- Not recommended for large projects, but useful for small examples.

```
<button onclick="alert('Hello!')">Click Me</button>
```

7. Multiple Events Example

- We can handle different events on the same element.

```
const box = document.getElementById("box");
box.addEventListener("mouseover", () => {
  box.style.backgroundColor = "lightblue";
});
box.addEventListener("mouseout", () => {
  box.style.backgroundColor = "white";
});
```

9. Key Takeaways

- ✓ Events make web pages interactive.
- ✓ Use `addEventListener()` to handle events.
- ✓ Event object provides useful information.
- ✓ We can remove event listeners using `removeEventListener()`.
- ✓ Common events: click, mouseover, keydown, change, submit, etc.
- ✓ Use `preventDefault()` to stop default browser behavior.
- ✓ Event handling is essential for dynamic web applications.

2. Common Events

Event	Description	Example Use Case
click	Triggered when element is clicked	Button click
dblclick	Triggered on double click	Open a file/folder
mouseover	Triggered when mouse hovers over an element	Show tooltip
mouseout	Triggered when mouse leaves an element	Hide tooltip
keydown	Triggered when a key is pressed	Detect key input
keyup	Triggered when a key is released	Form validation
change	Triggered when value of an input changes	Handle form input
submit	Triggered when a form is submitted	Process form data

4. Event Object

- When an event occurs, an event object is passed to the event handler.
- It contains useful information about the event.

```
btn.addEventListener("click", function(event) {
  console.log("Event type:", event.type); // click
  console.log("Target element:", event.target);
  console.log("Mouse X:", event.clientX);
  console.log("Mouse Y:", event.clientY);
});
```

6. Removing Event Listeners

- We can remove an event listener using `removeEventListener()`.

```
function handleClick() {
  console.log("Button clicked!");
}
btn.addEventListener("click", handleClick);
btn.removeEventListener("click", handleClick);
// listener removed
```

8. Practical Example (Form)

- Handling form submit event.

```
<form id="myForm">
  <input type="text" id="name" placeholder="Enter name">
  <button type="submit">Submit</button>
</form>
<script>
  document.getElementById("myForm").addEventListener(
    "submit", function(e) {
      e.preventDefault(); // prevents page reload
      const name = document.getElementById("name").value;
      console.log("Form submitted! Name:", name);
    });
</script>
```

16. Scope, Hoisting and Closure in JavaScript

Scope, hoisting and closure are important concepts in JavaScript. They help us understand how variables work, how code is executed and how functions can remember their environment.

1. Scope in JavaScript

- Scope determines the accessibility of variables in different parts of the code.
- JavaScript has three types of scope:
 - Global Scope
 - Function Scope
 - Block Scope (let, const)

2. Global Scope

- Variables declared outside any function or block have global scope.
- They can be accessed from anywhere in the code (after declaration).

```
let name = "Narayan";    // global scope
function show() {
    console.log(name);    // accessible here
}
show();    // Narayan
```

3. Function Scope

- Variables declared with var, let or const inside a function are only accessible within that function.

```
function greet() {
    let msg = "Hello";
    console.log(msg);    // Hello
}
greet();
// console.log(msg);    // Error: msg is not defined
```

4. Block Scope

- Variables declared with let and const inside a block {} are only accessible within that block.
- var does not have block scope (it is function scoped).

```
{
    let x = 10;
    const y = 20;
    console.log(x, y);    // 10 20
}
// console.log(x);    // Error
// console.log(y);    // Error
```

5. Hoisting

- Hoisting means variable and function declarations are moved to the top of their scope during execution.
- Only declarations are hoisted, not initializations.

Variable Hoisting	Function Hoisting
<pre>console.log(a); // undefined var a = 5; // let and const are also hoisted // but in Temporal Dead Zone (TDZ) console.log(b); // Error let b = 10;</pre>	<pre>sayHello(); // works function sayHello() { console.log("Hello"); }</pre>

6. Closure

- A closure is a function that remembers its lexical environment, even after the outer function has finished executing.
- Closures are useful for data hiding, counters, and maintaining state.

```
function outer() {
    let count = 0;
    return function() {
        count++;
        console.log("Count: " + count);
    }
}

const counter = outer();
counter();    // Count: 1
counter();    // Count: 2
```

The inner function remembers the variable count even after outer() has finished executing.

7. Key Takeaways

- ✓ Scope controls the accessibility of variables.
- ✓ Global, function and block scope are important.
- ✓ let and const are block scoped, var is function scoped.
- ✓ Hoisting moves declarations to the top, not initializations.
- ✓ Functions are fully hoisted.
- ✓ Closures allow functions to remember their lexical environment.
- ✓ These concepts are frequently asked in interviews.

17. Asynchronous JavaScript

JavaScript is single-threaded, which means it can do one task at a time. Asynchronous JavaScript allows us to perform time-consuming tasks without blocking the main thread, making the code non-blocking and more efficient.

1. Synchronous vs Asynchronous

- **Synchronous**
 - Executes code line by line.
 - Each operation waits for the previous one to complete.
 - Can block the execution if a task takes time.
- **Asynchronous**
 - Allows multiple operations to run without blocking the main thread.
 - Uses features like callbacks, promises and async/await.
 - Commonly used for tasks like API calls, file reading, etc.

2. Callback Function

- A callback is a function passed as an argument to another function and executed later.
- Used in asynchronous operations.
- Can lead to nested callbacks (callback hell).

```
function fetchData(callback) {
  setTimeout(() => {
    console.log("Data fetched"); // simulating
    callback(); // async task
  }, 2000);
}

fetchData(() => {
  console.log("Callback executed"); // runs after
  // 2 seconds
});
```

3. Callback Hell

- When multiple callbacks are nested inside each other, the code becomes hard to read and maintain.
- This is known as callback hell.

```
getData(() => {
  getMoreData(() => {
    getEvenMoreData(() => {
      console.log("Result"); // Hard to read
    }); // and maintain
  });
});
```

5. Async/Await

- Async/await is a modern way to handle promises.
- It makes asynchronous code look and behave like synchronous code.
- `async` is used before a function and `await` is used before a promise.

```
async function fetchData() {
  try {
    let response = await getData(); // cleaner and
    console.log(response); // easier to read
  } catch (error) {
    console.log("Error:", error);
  }
}

fetchData();
```

4. Promises

- A Promise is an object that represents the eventual completion (or failure) of an asynchronous operation.
- It can be in one of three states: pending, fulfilled or rejected.
- Helps to avoid callback hell.

```
const promise = new Promise((resolve, reject) => {
  let success = true;
  if (success) {
    resolve("Data loaded");
  } else {
    reject("Error occurred");
  }
});

promise
  .then((result) => console.log(result)) // Data loaded
  .catch((error) => console.log(error)); // Error occurred
```

6. Example: Fetch API

- The `fetch()` method is used to make HTTP requests and returns a promise.
- We can use `then/catch` or `async/await` to handle it.

```
async function getUser() {
  try {
    let res = await fetch("https://jsonplaceholder.typicode.com/users/1");
    let data = await res.json();
    console.log(data.name);
  } catch (error) {
    console.log("Error fetching data:", error); // fetches user data
  } // from an API
}

getUser();
```

7. Key Takeaways

- ✓ JavaScript is single-threaded, so asynchronous programming is important.
- ✓ Callbacks were the initial way but can lead to callback hell.
- ✓ Promises provide a better way to handle asynchronous operations.
- ✓ Async/await makes the code cleaner and easier to read.
- ✓ Asynchronous JavaScript is widely used for API calls, file handling, timers, etc.

18. JavaScript Interview Questions & Quick Revision

Some important interview questions and a quick revision of key concepts to strengthen your understanding of JavaScript.

1. Important Interview Questions

- Q1.** What is the difference between `let`, `const` and `var` ?
 Ans:
 - `var` : function scoped, can be redeclared.
 - `let` : block scoped, cannot be redeclared.
 - `const` : block scoped, cannot be reassigned.
- Q2.** What is a closure in JavaScript ?
 Ans: A closure is a function that remembers its lexical environment even after the outer function has finished executing.
- Q3.** What is the difference between `==` and `===` ?
 Ans: `==` compares values (with type coercion), while `===` compares both value and type (without type coercion).
- Q4.** What is hoisting ?
 Ans: Hoisting is JavaScript's behavior of moving declarations to the top of their scope during execution.
- Q5.** What is the event loop ?
 Ans: The event loop is a mechanism that handles asynchronous operations and executes callbacks when the call stack is empty.

3. Conceptual Questions

- Q11.** What is a Promise ?
 Ans: A Promise represents the eventual completion (or failure) of an asynchronous operation.
- Q12.** What is the difference between `call()`, `apply()` and `bind()` ?
 Ans: `call()` and `apply()` immediately invoke the function with a specified `this` value, while `bind()` returns a new function with the specified `this` value.
- Q13.** What is the use of `async/await` ?
 Ans: `async/await` makes asynchronous code look synchronous and easier to read, built on top of Promises.
- Q14.** What is the difference between `null` and `undefined` ?
 Ans: `null` is an intentional absence of value, while `undefined` means a variable is declared but not assigned a value.
- Q15.** What is the difference between a shallow copy and a deep copy ?
 Ans: A shallow copy copies only the reference (nested objects are shared), while a deep copy copies all levels (nested objects are not shared).

2. Output Based Questions

- Q6.** What will be the output?
`console.log(typeof null);`
 Ans: `object`
 (It's a known bug in JS)
- Q7.** What will be the output?
`console.log([] == false);`
 Ans: `true`
 (`[]` is converted to `"`)
- Q8.** What will be the output?
`console.log(0 == '0');`
`console.log(0 === '0');`
 Ans: `true`
`false`
- Q9.** What will be the output?
`console.log(() => {} instanceof Object);`
 Ans: `true`
 (Functions are objects)
- Q10.** What will be the output?
`console.log([1, 2, 3] + [4, 5]);`
 Ans: `1,2,34,5`
 (Arrays are converted to strings)

4. Code Snippets / Tricky Questions

- Q16.** Find the output.

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100);
}
```

 Ans: `3 3 3`
 (because `var` is function scoped)
- Q17.** Fix the above code using `let`.

```
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100);
}
```

 Ans: `0 1 2`
- Q18.** What will be the output?

```
function test() {
  console.log(a);
  var a = 10;
}
test();
```

 Ans: `undefined`
 (due to hoisting)

5. Quick Revision (Key Points)

- Variables: `var`, `let`, `const`
- Data Types (8 types)
- Operators (Arithmetic, Comparison, Logical)
- Type Conversion & Coercion
- Conditional Statements (`if`, `switch`)
- Loops (`for`, `while`, `do-while`)
- Functions (declaration, expression, arrow)
- Arrays (methods)
- Strings (methods)
- Objects (create, access, methods)
- Error Handling (`try`, `catch`)
- ES6+ Features (`let`, `const`, arrow, spread, rest, template literals)
- DOM Manipulation
- Events & Event Handling
- Scope, Hoisting, Closure
- Asynchronous JS (callback, promise, `async/await`)

6. Final Takeaways

- ✓ Understand core concepts, not just syntax.
- ✓ Practice writing and predicting code output.
- ✓ Focus on real-world examples (DOM, Events, Async JS).
- ✓ Be comfortable with ES6+ features.
- ✓ Solve more coding questions and implement small projects.
- ✓ These topics are frequently asked in interviews and are essential for modern web development.